

Revo 2 Ball Hold and In-Hand Movement Policies

CEM training, neural distillation, and repeatable evaluation

Cesar A. Pascal Vasquez

Manual 2

Purpose of this Manual

This manual converts the validated standalone Revo 2 Basic model into a quantitative policy-training experiment. A fixed wrist must acquire a randomized ball, retain it under gravity, reject a short disturbance, and optionally move it side to side inside the palm. The manual covers the environment, linear cross-entropy-method policy, neural policy, evaluation procedure, and interpretation of the saved results.

Experiment goal

Train a six-output closed-loop controller, verify it on unseen seeds, and distinguish successful ball retention from visually plausible finger closure.

Contents

1	Before You Begin	3
2	Experiment Overview	4
3	Environment Definition	5
3.1	Observation vector	5
3.2	Action vector	5
4	Episode Timeline and Randomization	6
5	Success Metrics and Experimental Controls	7
6	Choose Parameters and Write Your Policy	8
7	Baseline Code Package and Reading Order	9
8	Environment Code: Parameters, Reward, and Success	10
9	Linear Baseline Code and CEM Controls	11
10	Neural Baseline and a Student Modification	12
11	Linear Feedback Policy	13
12	Train the Hold Policy	14
13	Play and Evaluate the Hold Policy	15
14	Train the In-Hand Movement Policy	16
15	Train the Neural Hold Policy	17
16	Compare Linear and Neural Policies	18
17	Troubleshooting and Final Checklist	19
17.1	Training score does not improve	19
17.2	The ball launches at reset	19
17.3	Movement becomes slipping	19
17.4	Final checklist	19

1 Before You Begin

This manual can be completed without reading Manual 1. You need Python 3.10 or newer, MuJoCo, NumPy, the right or left Revo 2 XML/mesh package, and the editable Python files supplied in `editable_code/`. Work in simulation only.

Create the folder layout shown below, then copy the editable files from the source package into `scripts/`. Place the BrainCo model package under `vendor/` without renaming its mesh folders.

Required local layout

```
revo2_ball_lab/  
|-- requirements.txt  
|-- scripts/  
|   |-- revo2_grasp_env.py  
|   |-- train_grasp_policy.py  
|   |-- train_neural_grasp_policy.py  
|   |-- play_grasp_policy.py  
|-- vendor/Revo2_xml/xml_right/  
|-- vendor/Revo2_xml/xml_left/  
|-- policies/  
|-- reports/
```

Install and check the environment

```
cd "/path/to/revo2_ball_lab"  
python3 -m venv .venv  
.venv/bin/python -m pip install "mujoco>=3.3.6,<4" "numpy>=2,<3"  
.venv/bin/python -c "import mujoco, numpy; print('ready')"
```

Checkpoint 1

The check must print **ready**. Then run a three-iteration smoke test from Section 12. Do not begin a long search until the model loads and one candidate episode completes.

2 Experiment Overview

This experiment keeps the wrist fixed so the learning problem is limited to finger coordination. Each episode lasts 4.5 seconds. The ball begins near the open palm, the hand closes for 1.5 seconds, gravity ramps to full strength over 0.5 seconds, and a short randomized external force tests whether the grasp is robust.

Two related tasks are available:

- **Hold:** keep the ball close to a stationary target in the palm.
- **Move:** warm-start from the hold controller and track a small side-to-side target after the grasp is stable.

The experiment includes two policy families. The linear sigmoid policy provides an interpretable baseline and a safe teacher. The neural policy uses one hidden layer and is first distilled from that teacher before rollout optimization.

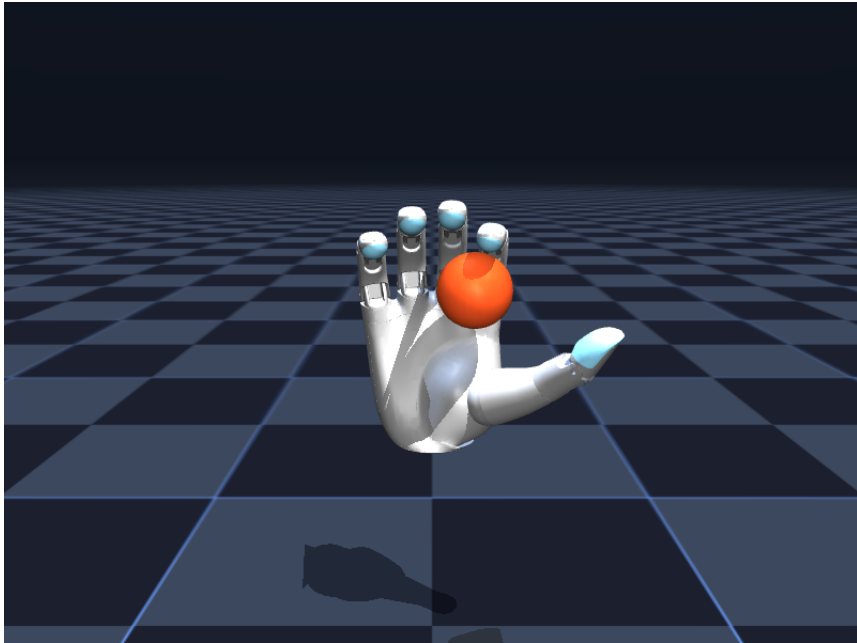


Figure 1: Fixed-wrist right hand and ball used by the hold experiment.

3 Environment Definition

The environment is implemented in `scripts/revo2_grasp_env.py` and depends only on NumPy and MuJoCo. It does not require Gymnasium. The control rate is 50 Hz, so the policy receives a new observation and produces a new action every 0.02 seconds.

3.1 Observation vector

The policy receives 19 normalized values.

Table 1: Observation groups

Group	Values	Meaning
Target error	3	Target position minus ball position
Ball linear velocity	3	Translational motion of the ball
Target velocity	3	Desired motion for the move task
Ball angular velocity	3	Rotation rate around three axes
Joint ratios	6	Normalized active hand positions
Gravity fraction	1	Progress of the gravity ramp
Total	19	State supplied to the policy

3.2 Action vector

The action is six values in the interval 0 to 1: thumb opposition, thumb flexion, and flexion for the four fingers. The environment maps these ratios into the official MuJoCo actuator ranges and slew-limits changes to prevent unrealistic instantaneous closure.

4 Episode Timeline and Randomization

The task is a closed-loop experiment, not a stored pose. The policy must react to the ball’s measured position and velocity throughout the episode.

Table 2: Nominal 4.5-second episode

Time	Event
0.00–1.50 s	Acquire the ball while gravity is disabled
1.50–2.00 s	Ramp gravity smoothly to 9.81 m/s^2
After 2.00 s	Maintain the grasp under full gravity
Short interval	Apply a 0.06 N disturbance for 0.12 s
After 2.25 s	For the move task, track a 6 mm sinusoidal target

The ball spawn uses a narrow nonzero distribution. A much wider distribution can place a 36 mm sphere inside the collision meshes and create nonphysical launch impulses before the controller can react. The disturbance direction changes with the seed.

```
1 class GraspTaskConfig:
2     episode_seconds = 4.5
3     control_hz = 50.0
4     acquire_seconds = 1.50
5     gravity_ramp_seconds = 0.50
6     move_amplitude = 0.006
7     move_period_seconds = 2.0
8     disturbance_force = 0.06
9     disturbance_seconds = 0.12
```

Listing 1: Core task parameters

5 Success Metrics and Experimental Controls

A policy is successful only when the ball remains in the hand through the evaluation interval. Visual finger closure alone is insufficient. The environment records score, success, dropped state, contact count, mean distance to the hold target, movement range, and movement tracking error.

- **Success rate** is the fraction of episodes that meet every task criterion.
- **Drop rate** is the fraction in which the ball leaves the valid grasp region.
- **Mean hold error** is the average ball-to-target distance after acquisition.
- **Contact count** measures the number of active hand-ball contacts.
- **Move range** measures controlled side-to-side ball displacement.

Use identical seeds when comparing policies. Otherwise, a controller may appear better because it received easier initial conditions or disturbance directions.

Minimum evidence for a valid comparison

Report the number of episodes, seed range, success rate, drop rate, mean hold error in millimeters, and final contact count. For movement, also report movement range and tracking RMSE.

6 Choose Parameters and Write Your Policy

The editable scripts are examples, not fixed answers. Select a computational budget and change one group of variables at a time.

Table 3: Recommended starting points and study ranges

Parameter	Start	Study range	Tradeoff
Control rate	50 Hz	25–100 Hz	Faster feedback costs more simulation steps
Disturbance force	0.06 N	0–0.12 N	Tests grasp robustness
Move amplitude	6 mm	2–12 mm	Larger motion increases slip risk
CEM population	28	12–80	More candidates improve search but cost time
Episodes/candidate	2	1–8	More seeds reduce noisy ranking
Elite fraction	0.20	0.10–0.30	Controls how selective each update is
Neural hidden size	16	8–64	Adds capacity and parameters

Any custom policy only needs to accept 19 observations and return six values in $[0, 1]$. Save this example as `scripts/student_policy.py`, then modify the weights or replace the class with your own network.

```
1 import numpy as np
2
3 class StudentPolicy:
4     def __init__(self):
5         power = np.array([.82, .88, .84, .88, .90, .92])
6         self.W = np.zeros((6, 19))
7         self.b = np.log(power / (1.0 - power))
8
9     def __call__(self, observation):
10        z = self.W @ np.asarray(observation) + self.b
11        return 1.0 / (1.0 + np.exp(-z))
```

Listing 2: Minimal policy interface students can replace

Evaluate it with `env.rollout(StudentPolicy(), seed=1200)`. For a fair study, choose training seeds before optimization and a different consecutive seed block for final evaluation.

Your experimental question

Write one question before coding, for example: “Does doubling population improve unseen-seed success more than doubling episodes per candidate?” Independent variable: _____.
Fixed variables: _____.

7 Baseline Code Package and Reading Order

The folder `editable_code/` contains the complete baseline Python source. Copy its files into the local `scripts/` folder before running them. Preserve a clean copy in version control or in an untouched folder.

Table 4: Baseline source files

File	Read or edit it when you want to
<code>revo2_grasp_env.py</code>	Change task timing, disturbances, observations, reward, success, or randomization
<code>train_grasp_policy.py</code>	Understand or modify the linear sigmoid policy and CEM search
<code>neural_grasp_policy.py</code>	Change hidden width, activation, forward pass, or gradient calculation
<code>train_neural_grasp_policy.py</code>	Change teacher distillation and neural CEM settings
<code>play_grasp_policy.py</code>	Load a policy, choose seeds, open the viewer, or run headless episodes
<code>compare_grasp_policies.py</code>	Evaluate two saved policies on the same seed block and export CSV/J-SON

The normal call sequence is:

1. a training script creates `GraspTaskConfig`;
2. `Revo2BallGraspEnv` loads the XML and resets a seeded episode;
3. the environment builds a 19-value observation;
4. `policy(observation)` returns six normalized commands;
5. `env.step(action)` advances physics and calculates reward;
6. `env.metrics()` decides success and reports errors;
7. training saves weights to an NPZ file;
8. playback or comparison reloads the NPZ on unseen seeds.

Edit one layer at a time

First modify either the environment or the policy, not both. Otherwise a score change cannot be attributed to one cause. Save the exact code and terminal command with every result.

8 Environment Code: Parameters, Reward, and Success

Open `revo2_grasp_env.py`. Search for class `GraspTaskConfig`; this dataclass is the main experiment control panel.

```
1 @dataclass(frozen=True)
2 class GraspTaskConfig:
3     task: str = "hold"
4     episode_seconds: float = 4.5
5     control_hz: float = 50.0
6     acquire_seconds: float = 1.50
7     gravity_ramp_seconds: float = 0.50
8     actuator_slew_seconds: float = 1.50
9     move_amplitude: float = 0.006
10    move_period_seconds: float = 2.0
11    disturbance_force: float = 0.06
12    disturbance_seconds: float = 0.12
```

Listing 3: Baseline task configuration

Table 5: Where environment behavior is defined

Search for	What students may change
<code>GraspTaskConfig</code>	Timing, rate, motion amplitude, disturbance magnitude and duration
<code>reset</code>	Ball spawn distribution, episode seed, and initial state
<code>observation</code>	The 19 features and their order; update every policy if this changes
<code>_physics_step</code>	Gravity ramp, actuator slew, and disturbance application
<code>_reward</code>	Dense optimization terms and drop penalty
<code>metrics</code>	Success thresholds, contact minimum, movement range, tracking RMSE
<code>rollout</code>	The policy–environment loop and optional callback

Change success thresholds only before training and state them in the report. A looser threshold can increase reported success without improving behavior.

9 Linear Baseline Code and CEM Controls

Open `train_grasp_policy.py`. `LinearGraspPolicy.__call__()` reshapes 120 parameters into a 6×20 matrix, appends a bias feature to the 19 observations, and applies the sigmoid. `initial_parameters()` centers the first population around a power grasp.

The `main()` loop samples candidates, calls `evaluate()`, sorts scores, and updates the distribution from the elite set.

```
1 elite_indices = np.argsort(scores)[-elite_count:]
2 elites = candidates[elite_indices]
3 elite_mean = np.mean(elites, axis=0)
4 elite_std = np.std(elites, axis=0)
5
6 mean = 0.25 * mean + 0.75 * elite_mean
7 std = np.clip(0.35 * std + 0.65 * elite_std,
8               0.012, 0.45)
```

Listing 4: The editable CEM update in the baseline

Most training choices do not require editing Python. They are arguments defined in `parse_args()`.

Table 6: Linear policy command-line controls

Flag	Default	Meaning
<code>--iterations</code>	10	Number of sample-select-update cycles
<code>--population</code>	28	Candidates evaluated per cycle
<code>--episodes</code>	2	Randomized episodes used to score each candidate
<code>--elite-fraction</code>	0.20	Fraction retained for distribution update
<code>--seed</code>	7	Reproduces candidate sampling and episode schedule
<code>--initial-policy</code>	none	Warm-starts movement from a saved policy
<code>--output</code>	task path	Changes where the NPZ is saved

Edit the four coefficients in the update only if studying CEM itself. Record them because they are not stored as command-line flags.

10 Neural Baseline and a Student Modification

The neural implementation is split deliberately. `neural_grasp_policy.py` defines the network; `train_neural_grasp_policy.py` defines distillation, search, arguments, saving, and evaluation.

In `NeuralGraspPolicy.forward_batch()`, the baseline applies one tanh hidden layer and a sigmoid output. To change activation, edit both the forward expression and its derivative in `binary_cross_entropy_gradient()`.

Table 7: Neural parameters and exact locations

Choice	Location	How to change it
Hidden width	<code>--hidden-size</code>	Pass 8–64 in the training command
Distillation data	<code>--distill-episodes</code>	Changes teacher trajectories
Distillation updates	<code>--distill-steps</code>	Changes supervised fitting time
Neural CEM budget	<code>--iterations/</code> <code>--population/--episodes</code>	Changes rollout search cost
Distillation LR	<code>distill_teacher</code>	Edit <code>learning_rate</code> default
Exploration scale	<code>exploration_std</code>	Edit per-layer search standard deviations
CEM smoothing	<code>near elite_mean</code>	Edit 0.30/0.70 and 0.40/0.60 coefficients

Example: test whether 32 hidden units improve unseen-seed hold error.

Baseline, edited training, and evaluation

```
.venv/bin/python scripts/train_neural_grasp_policy.py \  
  --hidden-size 16 --seed 17 \  
  --output policies/neural_h16.npz  
.venv/bin/python scripts/train_neural_grasp_policy.py \  
  --hidden-size 32 --seed 17 \  
  --output policies/neural_h32.npz  
.venv/bin/python scripts/play_grasp_policy.py \  
  --policy policies/neural_h32.npz \  
  --headless --episodes 30 --seed 30000
```

Keep the seed and all other flags identical. Compare success, drops, hold error, and contacts; do not choose a network from one visually successful episode.

11 Linear Feedback Policy

The baseline controller has six sigmoid output heads. Each output uses the 19 observations plus one bias, so the complete policy has

$$6(19 + 1) = 120$$

trainable parameters. For observation vector o , output i is

$$a_i = \sigma(b_i + w_i^T o),$$

where σ is the logistic sigmoid. This structure is small enough for gradient-free optimization and easy to inspect.

The initial mean is based on the power grasp. Cross-entropy method (CEM) samples a population around the current mean, evaluates each candidate, retains the best 20 percent, and moves the search distribution toward those elite candidates.

```
1 elite_indices = np.argsort(scores)[-elite_count:]
2 elites = candidates[elite_indices]
3
4 elite_mean = np.mean(elites, axis=0)
5 elite_std = np.std(elites, axis=0)
6
7 mean = 0.25 * mean + 0.75 * elite_mean
8 std = np.clip(0.35 * std + 0.65 * elite_std,
9               0.012, 0.45)
```

Listing 5: Core CEM distribution update

12 Train the Hold Policy

Complete the standalone validation from Manual 1 before running policy search. Then train the default right-hand hold controller.

Default hold training

```
cd "/path/to/brainco_revo2_mujoco"
.venv/bin/python scripts/train_grasp_policy.py \
  --hand right --task hold
```

Default parameters are 10 CEM iterations, 28 candidates per iteration, two episodes per candidate, elite fraction 0.20, and random seed 7. The policy is saved to `policies/revo2_right_hold.npz`; a JSON file beside it stores the training metadata.

Use a short run only to check the pipeline.

Smoke test

```
.venv/bin/python scripts/train_grasp_policy.py \
  --hand right --task hold --quick \
  --output policies/revo2_right_hold_quick.npz
```

Do not report the smoke test as the final policy

The quick configuration uses only three iterations, 12 candidates, and one episode per candidate. It verifies execution but is not a reliable robustness result.

13 Play and Evaluate the Hold Policy

Open the interactive result after training.

Interactive playback

```
.venv/bin/python scripts/play_grasp_policy.py \  
--policy policies/revo2_right_hold.npz
```

Run a repeatable headless evaluation using fresh episodes.

Ten-episode evaluation

```
.venv/bin/python scripts/play_grasp_policy.py \  
--policy policies/revo2_right_hold.npz \  
--headless --episodes 10
```

Watch for these stages: open approach, gradual closure, gravity ramp, stable contact, external disturbance, and final retention. A controller that squeezes the ball out of the palm should fail even if it briefly creates many contacts.

The saved baseline was later tested on 23 identical comparison seeds. It achieved 23 successes, zero drops, approximately 5.64 mm mean hold error, and seven mean final contacts. These values describe the fixed-wrist simulation and must not be treated as physical-hand accuracy.

14 Train the In-Hand Movement Policy

The movement objective is harder because the hand must preserve a grasp while creating a small controlled displacement. Warm-start from the trained hold policy rather than beginning from a random population.

Warm-start movement training

```
.venv/bin/python scripts/train_grasp_policy.py \  
  --hand right --task move \  
  --initial-policy policies/revo2_right_hold.npz \  
  --output policies/revo2_right_move.npz
```

Play the result with:

Movement playback

```
.venv/bin/python scripts/play_grasp_policy.py \  
  --policy policies/revo2_right_move.npz
```

The target oscillates by 6 mm along one palm axis. The experiment therefore studies in-hand repositioning, not arm motion through space. The included policy retained the ball in 20 of 20 verification episodes and produced about 2.2 mm of measured side-to-side ball motion.

Expected result

A movement policy should be judged on both retention and tracking. A large ball range is not useful if it comes from slipping or a near-drop.

15 Train the Neural Hold Policy

The neural controller has 19 inputs, one hidden layer of 16 tanh units, and six sigmoid outputs. It contains 422 parameters:

$$19(16) + 16 + 16(6) + 6 = 422.$$

For safer initialization, the network first imitates the trained linear controller. Ten teacher episodes are doubled with observation jitter, and 700 Adam steps minimize binary cross-entropy. CEM then fine-tunes network parameters using eight iterations, 28 candidates, and three episodes per candidate.

Neural training

```
.venv/bin/python scripts/train_neural_grasp_policy.py \  
--hand right --task hold \  
--initial-policy policies/revo2_right_hold.npz
```

Neural playback

```
.venv/bin/python scripts/play_grasp_policy.py \  
--policy policies/revo2_right_hold_neural.npz
```

Distillation gives the network a competent starting behavior; rollout optimization then searches for corrections that improve the true MuJoCo score.

16 Compare Linear and Neural Policies

Run both controllers on the same 23 seeds and save machine-readable reports.

Paired evaluation

```
.venv/bin/python scripts/compare_grasp_policies.py --episodes 23
```

Table 8: Recorded paired hold-policy comparison

Metric	Linear	Neural
Episodes	23	23
Success rate	100%	100%
Drop rate	0%	0%
Mean hold error	5.64 mm	5.35 mm
Mean final contacts	7.0	6.0
Mean score	2.9168	2.9212

The result shows that both policies solved this evaluation set. The neural network had a slightly smaller mean hold error and slightly higher score, but the difference is small and should not be generalized without more seeds or confidence intervals. The comparison files are `reports/linear_vs_neural_hold.json` and `reports/linear_vs_neural_hold.csv`.

17 Troubleshooting and Final Checklist

17.1 Training score does not improve

Confirm that candidate seeds change between iterations, the initial power grasp is valid, and the reward penalizes drops. Increase population size before increasing parameter variance.

17.2 The ball launches at reset

Inspect initial collision penetration. Do not widen the spawn distribution until the mesh geometry has been validated.

17.3 Movement becomes slipping

Reduce movement exploration, warm-start from the verified hold policy, and report tracking RMSE together with ball range.

17.4 Final checklist

- ☐ The same environment and seeds are used for policy comparison.
- ☐ The saved metadata records training parameters and seed.
- ☐ Evaluation uses episodes not used during CEM selection.
- ☐ Success, drops, hold error, contacts, and score are reported.
- ☐ Simulation results are not described as physical-hand performance.

References

1. BrainCo Hand. Revo 2 product parameters.
<https://www.brainco-hz.com/docs/revolimb-hand-dev/en/revo2/parameters.html>
2. DeepMind. MuJoCo computation and simulation documentation.
<https://mujoco.readthedocs.io/en/stable/computation/index.html>
3. Rubinstein and Kroese. The Cross-Entropy Method.
<https://doi.org/10.1007/978-1-4757-4321-0>
4. Kingma and Ba. Adam: A Method for Stochastic Optimization.
<https://arxiv.org/abs/1412.6980>